

CONSENSUS AND FORKS LAB

Group Member:

Ontshiametse Aphane - 218034515
Ntumeleng Singo - 217073026
Lindokuhle Nsibande - 226250686
Siphokazi Magaga - 218010124
Thapelo Raseu - 226226418

PART A

Simulating three nodes A, B and C

Three Node objects, named A, B and C, are created in Section 3 of the notebook. Each node starts by holding a deep copy of the same genesis block, so at the start of the demo all three nodes agree on one shared tip. Each node stores its own chain as a plain Python list, which means a node's chain can be extended independently of the others, allowing the nodes to diverge.

This simulation used is the in-process, this implies all three nodes exist as Python objects inside the same notebook kernel rather than as separate operating system processes.

```
A: height=0 tip=0034d79842... valid=True work=2
B: height=0 tip=0034d79842... valid=True work=2
C: height=0 tip=0034d79842... valid=True work=2
Shared tip? True
```

The image above shows the 3 blocks A, B and C having a shared genesis tip.

Creating a fork and resolving it

Creating the fork

In Section 4 of the notebook, node A mines a block at height 1 containing a R10,000 payment to Alice, and node B independently mines a different block at height 1 containing a R10,000 payment to Bob. Both blocks extend the same genesis parent and both pass `verify_chain`, so both are valid on their own and Node C is left at genesis.

```
A and B tips differ? True
Both extensions valid? True | True
Same parent (genesis)? True
A: height=1 tip=00ed0b3d99... valid=True work=4
B: height=1 tip=00949316e9... valid=True work=4
C: height=0 tip=0034d79842... valid=True work=2
```

Board sketch:

```
A: G -> 1_A (pay-alice)
B: G -> 1_B (pay-bob)
C: G
```

Choosing a resolution rule

Section 5 compares two possible scoring rules: `chain_length`, which simply counts blocks (the longest-chain rule), and `chain_work`, which sums each block's recorded difficulty (the heaviest-work

rule). Under uniform classroom difficulty the two rules rank chains identically, because work equals length multiplied by the constant difficulty. They only diverge if difficulty varies between blocks, in which case a short, hard chain can beat a long, easy chain. `chain_work`, is the one implemented inside `resolve_fork`, because it also behaves correctly in the more general case where difficulty is not constant.

Resolving the fork

In Section 6, first removes any chain that fails `verify_chain`, then picks the chain, or chains, with the highest `chain_work` score. If exactly one chain has the highest score, that chain wins outright. `sync_nodes`, in Section 7, calls `resolve_fork` once across every node in a list and overwrites each node's local chain with a deep copy of the winner, so after `sync_nodes` runs, all nodes share one canonical tip.

Before sync:

```
A: height=2 tip=003839fc75... valid=True work=6
B: height=1 tip=00949316e9... valid=True work=4
C: height=0 tip=0034d79842... valid=True work=2
```

After sync (heaviest / longest under uniform difficulty):

```
A: height=2 tip=003839fc75... valid=True work=6
B: height=2 tip=003839fc75... valid=True work=6
C: height=2 tip=003839fc75... valid=True work=6
Heights before: {'A': 2, 'B': 1, 'C': 0}
Shared tip after: 003839fc757b91dc ...
Canonical payment id: pay-alice
```

The above image shows three nodes before sync, they have different heights and tips and after sync they all have the same height and identical tips. This is shows chain A is the winner and the rest of the chains were overwritten with the winner chain.

Documenting the required edge cases

Ties

A tie happens when two or more valid chains reach the same `chain_work` score, this is the case for blocks A and B which both mine a block at height 1. We implemented the deterministic tie-breaker method, where the chain whose tip block has the lexicographically smaller hash wins. This is implemented in `tie_break` and called from inside `resolve_fork` whenever more than one chain shares the maximum score.

Section 6 also demonstrates the companion rule that a longer but invalid chain must never win a tie or any other comparison, by constructing a chain with a tampered hash and confirming that `resolve_fork` rejects it in favour of a shorter, valid chain.

```
Invalid longer chain rejected; winner height: 1
Winner valid? True
```

Delayed block arrival

A delayed block is one that reaches a node later than it was produced, for example because of network latency or message reordering. Section 8 of the notebook shows node C first hears only B's shorter

chain and adopts it as its local view, then later hears A's longer chain, using the heavier chain tie breaker method, it reorganizes onto it, discarding B's block from its canonical view.

```
C start: C: height=0 tip=0034d79842... valid=True work=2
=====
C after hearing B:
=====
C after hearing B: C: height=1 tip=00949316e9... valid=True work=4
  local payment id: pay-bob
=====
C after hearing A (heavier):
=====
C after hearing A: C: height=2 tip=003839fc75... valid=True work=6
  reorganisation occurred? True
  local payment id now: pay-alice
```

The image above shows C first adopting B's tip and then reorganizing onto A's heavier chain once it hears it.

Confirmation depth for a R10,000 payment

Confirmation depth is computed by `find_tx_height`, which locates the block index `h` that contains a given transaction id, and `confirmations`, which returns $k = H - h + 1$, where `H` is the index of the current chain tip. A transaction that is not present anywhere on the chain, such as `pay-bob` after node A's chain wins the fork, is reported as having 0 confirmations.

```
'pay-alice' height: 1
'pay-alice' confirmations at current tip: 2
'pay-bob' confirmations (orphaned): 0
After padding to height 5 -> confirmations: 5
Missing tx -> 0
```

The above image shows the confirmation depth of `pay-alice` before and after padding the chain, the missing transaction id returns 0 confirmations

Section 10 applies this to Alice's R10,000 remittance directly. The notebook checks the `pay-alice` transaction against three illustrative policies: a coffee-shop style policy requiring only $k = 1$, a stricter class policy requiring $k = 3$, and an OTC-desk style policy requiring $k = 6$. The report's position, to be expanded on by the group, is that a larger required k lowers the risk that a payment is reversed by a chain reorganization, at the cost of a slower settlement time, and that a sensible choice of k should be justified against the cost of acquiring enough hash power to actually rewrite that many blocks, not chosen arbitrarily.

```
pay-alice confirmations now: 2
  coffee-shop (k>=1): True
  class policy (k>=3): False
  OTC desk (k>=6): False
```

Qualitative check for the report:

```
Ticket size: R10,000
Toy difficulty: 2 (not production security)
Chosen classroom k: 3
Justification: a reorganisation must redo k blocks of work;
               a larger ticket size usually justifies a larger k.
```

PART B - TEXTBOOK DEEP-DIVES

Chapter 7 - Advanced Transactions and Merkle Trees

Central mechanism. Bitcoin transactions do more than move a balance from one named account to another. Each transaction spends earlier outputs and creates new ones, with locking conditions that determine who may spend the value next. The transaction identifier commits to the serialised transaction data, while the Merkle root in the block header commits to all transactions included in that block. A Merkle tree is built by hashing transaction identifiers in pairs and repeating the process until only one root remains. If any transaction is altered, that change moves up through the tree and produces a different root. The header therefore contains a compact integrity commitment rather than a complete copy of every transaction (Antonopoulos and Harding, 2023, pp. 119-142, 245-262).

Merkle proofs make this structure useful for simplified payment verification (SPV). A lightweight client does not need the whole block to check whether a transaction was included. Instead, it combines the transaction hash with a small set of neighbouring hashes and reconstructs the Merkle root recorded in the header. Because the proof grows logarithmically rather than linearly, it remains relatively small even when a block contains many transactions. This convenience has a trade-off: an SPV client relies on the header chain with the greatest cumulative work and does not enforce every consensus rule as independently as a full node. Inclusion is therefore evidence that a transaction appears in a particular block, not a guarantee that the payment is final (Antonopoulos and Harding, 2023, pp. 245-262). Signatures confirm spending authority, but settlement also depends on the relevant locking conditions and on whether the block remains on the selected chain (Antonopoulos and Harding, 2023, pp. 143-202).

Worked example. Consider a block containing eight transactions, T1-T8, where T6 represents a R10 000 remittance. The first layer contains H1 = hash(T1) through H8 = hash(T8). These hashes are paired to form H12, H34, H56 and H78, followed by H1234 and H5678. The root is hash(H1234||H5678). To verify T6, the recipient needs the hash of T6 together with only H5, H78 and H1234. These values are enough to reconstruct H56, then H5678 and finally the root. The recipient does not need the other seven transactions. If the amount in T6 is changed, its hash also changes, so the reconstructed root will no longer match the root stored in the block header.

Repo connection. This mechanism has direct implications for the SimpleBlockchain stack. The transaction.py module should serialise transaction data in a deterministic, canonical form before hashing it; otherwise, two equivalent transactions may produce different identifiers. The block.py module should commit to the transaction root in the header, while blockchain.py should recompute and verify that root before accepting a block. The A7 consensus.py logic is still necessary because a valid Merkle proof confirms membership in one block, but cannot show that the block will survive fork choice.

Change to the remittance risk memo. The memo should separate four stages that are often treated as if they were the same: signed, propagated, included and economically settled. For a R10 000 off-ramp, a valid inclusion proof can support efficient monitoring, but it should not by itself trigger an irreversible payout. The provider should verify the proof against a recognised block header, check the spending conditions, monitor possible reorganisations and wait for the approved number of confirmations. It should also retain the full transaction record for customer due diligence, sanctions screening, exchange-control evidence and dispute resolution. In short, a Merkle proof establishes cryptographic inclusion; it does not establish the payer's identity, the legitimacy of the payment purpose, the correct foreign-exchange treatment or finality of the fiat payout.

Chapter 12 - Mining and Consensus

Central mechanism. Mining is better understood as Bitcoin's decentralised clearing process than simply as a way of creating new coins. Miners collect valid transactions, build a candidate block and vary the nonce or other mutable fields until the block-header hash falls below the current target. Other nodes then verify the transactions, the link to the previous block and the proof-of-work. If two valid tips exist, nodes converge on the chain with the greatest cumulative proof-of-work. The block subsidy and transaction fees reward miners for committing computational resources, linking their private incentives to the security of the network (Antonopoulos and Harding, 2023, pp. 263-302).

Even so, consensus provides probabilistic rather than immediate finality. A newly mined payment can still be displaced if a competing branch accumulates more work. Each additional block built above the payment reduces the chance of reversal, but does not remove it completely. Difficulty retargeting helps to stabilise block production when total network hash power changes. Bitcoin compares the actual time taken to mine 2 016 blocks with the target period of $2\,016 \times 10$ minutes and adjusts the proof-of-work target within protocol limits. Faster production leads to higher difficulty, while slower production leads to lower difficulty (Antonopoulos and Harding, 2023, pp. 263-302). Confirmation time is therefore uncertain in practice rather than a guaranteed service interval.

Worked example. Suppose 2 016 blocks are mined in 12 days instead of the intended 14 days. Using the simplified ratio, new difficulty = old difficulty $\times 14/12$, so difficulty increases by approximately 16.7%. If the previous toy difficulty were 3.00, the adjusted value would be about 3.50. This change moves the expected block interval back towards ten minutes, assuming hash power remains broadly stable after the retarget. In the course's toy miner, requiring three leading hexadecimal zeroes gives an expected search cost of $16^3 = 4\,096$ hash trials. If block 2 in a five-block chain is edited, its hash changes and blocks 3-5 continue to point to obsolete predecessor hashes. An attacker would therefore have to rebuild the altered suffix while also trying to catch the honest chain's continuing cumulative work.

The incentive problem makes this mechanism less straightforward than it first appears. A miner or coalition may withhold a newly discovered block and release it strategically, hoping that honest miners will build on a branch that later becomes stale. Whether this behaviour is profitable depends on the coalition's share of hash power, its propagation advantage and the way miners choose between branches of equal work. Having less than 51% of the network's hash power is therefore not automatically harmless, even though majority control remains central to sustained chain domination (Eyal and Sirer, 2014, pp. 436-454). Pool concentration and network latency should consequently be treated as governance and operational risks, not merely as technical details.

Repo connection. The A6 mining.py module demonstrates nonce search, difficulty and coinbase construction, while the A7 consensus.py module implements fork choice and confirmation counting. A more production-oriented version should compare cumulative work rather than chain length alone, reject invalid coinbase rewards, recompute every block hash and verify difficulty transitions. Its tests should also simulate two miners discovering blocks at nearly the same time and trace what happens to transactions returned to the mempool after a reorganisation.

Change to the remittance risk memo. The confirmation policy for a R10 000 off-ramp should be risk-based rather than expressed as one fixed number for every payment. The decision should consider the transaction value, observed congestion, counterparty history, the selected chain's work advantage, mining-pool concentration and recent stale-block conditions. A low-risk customer might receive a provisional credit after one confirmation, but an irreversible external payout should require a stronger threshold and an exception process for unusual reorganisations or sharp changes in hash rate. The memo should also make clear that confirmations address blockchain reversal risk only. They do not replace liquidity buffers, fraud controls, customer verification, sanctions screening or compliance with applicable payment and exchange-control requirements.

PART C - NAKAMOTO (2008)

Problem Framing

Nakamoto (2008) discusses double spending in digital transactions. Unlike tangible cash, digital data can be copied, and a payer can use the same digital asset multiple times. Digital signatures can verify that a transaction was authorized by the owner but not independently verify that the same funds have not been used before in a different transaction. Traditional payment systems are based on a trusted financial intermediary who keeps the official record of transactions. Nakamoto proposes using as peer-to-peer network to have public transaction broadcasting, timestamping, cryptographic hashing and proof-of-work (PoW) to establish a common transaction history without any central authority to order transactions (Nakamoto, 2008).

From the perspective of regulated remittance or financial market infrastructure (FMI), the parties in the industry are not just the sender and the recipient. These include remittance service providers, commercial banks, settlement entities, regulators, central banks, miners or network operators, etc. These are the stakeholders who are connected not only with double spending but also with settlement finality, operational resilience, legal certainty, liquidity risk, governance and consumer protection. And so Nakamoto may tackle the settlement integrity issue, but the paper does not address the entire range of risks that regulated payment systems are prone to.

Mechanism Audit

Nakamoto's method consists of digital signatures, transaction dissemination, hash-linked timestamps, proof-of-work and a selection rule based on the most work completed. Digital signatures offer the right to execute transactions. Transactions are organised in blocks, and each block is associated with the hash of the previous block. Miners update a nonce and compute a hash using SHA-256 function and builds a hash that meets the difficulty criterion of the network. As each block is linked to its predecessor, changing a transaction will change the hash of the next block and will therefore break the link to the previous block. As a result, an attacker must redo the proof-of-work for the modified block and of all the subsequent blocks and the valid network continues to extend the valid chain (Nakamoto, 2008).

The main mathematical analysis of the paper is presented in Section 11, where Nakamoto defines:

- p = probability an honest node finds the next block
- q = probability the attacker finds the next block
- z = number of blocks by which the attacker is behind
- $q(z)$ = probability the attacker will ever catch up from z block behind

Nakamoto characterizes the competition between the honest network and the attacker by a binomial random walk like the Gambler's Ruin scenario. The probability that the attacker ever catches up is given by:

$$q(z) = \begin{cases} 1 & \text{if } p \leq q \\ \left(\frac{q}{p}\right)^z & \text{if } p > q \end{cases}$$

If the attacker has at least as much computational power as the honest network, $q \geq p$, then the model says that catching up is inevitable. If $p > q$, then the chance of the attacker catching up is small and it gets smaller as z increases.

In case the double spending, Nakamoto also suggest that the number of blocks generated by the attacker while the honest network generates z can be approximated with a Poisson distribution. The expected number of blocks generated by the attacker is:

$$\lambda = z \frac{q}{p}$$

Nakamoto combines the Poisson probabilities with the catch-up probabilities to calculate the probability of a successful reversal. For example, when $q=0.1$, his calculation gives $p=0.0002428$ after six confirmations; importantly, this is not zero. The results thus support the idea that an increase in confirmations may make it less likely to reverse transactions if the honest computational power is predominant. That said, this does not mean that the reversal is impossible.

The model is based on several important assumptions. It needs to be true that honest nodes collectively have more computing power than any colluding attackers. It needs to also have messages spread in the network to such a degree that the honest nodes eventually agree on a common history, although Nakamoto explicitly describes broadcasting as “best effort”. When two valid blocks are broadcasted simultaneous at the same time, creating temporary branches, nodes initially extend the branch they receive first, and later follow the branch accumulating greater PoW. Nakamoto also notes that economic incentives are what motivate miners to follow protocol rather than undermine it.

Most of these assumptions are either not proven in general or not in the paper. The probability of an attacker catching up is calculated mathematically and numerical probabilities of attacks are calculated bas on the model. In contrast, network propagation, miner behaviour, compatibility of incentives and actual convergence are mostly assumed or discussed and not verified empirically in the real world. Decker and Wattenhofer (2013) showed that network propagation delay can lead to blockchain forks and this shows that network conditions are necessary for proof-of-work processes.

Evidence and Limits

The main drawback is that Nakamoto’s evidence is much more analytical and not empirical. In Section 11, attack probabilities are computed based on the values of p , q and z , but the paper does not draw on an existing production network and does not draw in fields data about latency, mining concentration, reorganisation frequency, outages or adversarial actions. The calculations clearly show sensitivity of the results: to achieve a probability of less than 0.001 ($P < 0.001$) requires only 5 confirmations when ($q=0.10$), 24 when ($q=0.30$), 89 when ($q=0.40$) and 340 when ($p=0.45$). Thus, waiting for confirmations is not a good risk management strategy, since the time it takes to wait for confirmed results seems to depend a lot on the estimated adversarial share.

A retail remittance of R10,000 would require a financial market infrastructure (FMI) analysis of the distribution of block production and confirmation time in the system, the number of stale blocks and chain reorganizations, transaction fees and throughput, block propagation delay, mining pool allocation of the network and cost/practicality of achieving the hash power for executing a double spend. Decker and Wattenhofer (2013) provide empirical evidence that propagation delay is important and Gervais et al. (2016) show that the security and performance of PoW is influenced by propagation speed, block size, block generation interval and adversarial strategy.

Importantly, a remittance of R10,000 does not determine a minimum number of confirmations. A regulated provider should set some reasonable level of reversal risk and should evaluate it with respect

to fraud risk, delay on confirmations and the cost of possible attacks. The proof of low reorganization rates, decentralized mining power, and performance in challenging network conditions would serve to support Nakamoto's case and the failure of the system to be consistently concentrated and propagation time long or economically viable reorganizations would weaken it.

Comparison with BFT/FMI Finality

A useful comparison can be made with Castro and Liskov's (1999) Practical Byzantine Fault Tolerance (PBFT) that deals with Byzantine behaviour among a specific group of replicas through a quorum-based consensus rather than through an open PoW competition. Not only do they implement the protocol, but they also experimentally test its performance in an asynchronous network.

The main difference between payments is finality. Nakamoto provides a probabilistic endgame: if z increases, the likelihood of reversal becomes less likely to be reversed as $q < p$, but his model fails to give a finite number of confirmations when the probability of an attack becomes negligible. On the other hand, BFT-style protocols can consider a transaction to be finalized when all the necessary conditions for agreement are satisfied based on the assumptions about faults and network.

This is especially significant for regulated infrastructures. CPMI-IOSCO Principle 8 states that a Financial Market Infrastructure (FMI) must clearly articulate when settlement is final, ensuring complete settlement by value date and when payment can no longer be reversed. To achieve Nakamoto's technical confirmation confidence, additional legal and operational frameworks need to be in place to transform probabilistic assurance into settled finality.

Open Question

Two research questions emerge directly from these limitations.

Research Question 1: For a retail remittance of R10,000, what is the necessary number of PoW confirmations to maintain the likelihood of a successful double-spend below a threshold set by regulators, considering variations in attacker hash power, propagation latency, and mining concentration?

This inquiry aims to build upon Nakamoto's $(P(q,z))$ analysis by substituting theoretical parameters with actual network data. Simulations could integrate observed propagation distributions alongside different q values and assess the correlation between predicted and actual reorganization frequencies. This investigation is driven by findings that indicate propagation conditions significantly influence PoW security (Decker & Wattenhofer, 2013; Gervais et al., 2016).

Research Question 2: In the context of identical R10,000 retail-payment workloads and network disruptions, what are the differences in time-to-finality, transaction throughput, and the likelihood of settlement failure or reversal between PoW and permissioned BFT?

A controlled experiment could introduce latency shocks, node failures, and Byzantine behaviour to both systems and evaluate the outcomes of settlements. This would examine whether the probabilistic security framework of PoW meets acceptable performance standards for regulated payments in comparison to a quorum-based BFT architecture (Castro & Liskov, 1999).

Verdict

Recommendation: Revise before consideration as a framework for regulated remittance systems.

Nakamoto significantly advances methodology by demonstrating how digital signatures, public transaction ordering, hash-linked blocks, and proof of work (PoW) can prevent double spending without reliance on a central authority for transaction ordering. The calculations in Section 11 provide some quantitative insights that when $q < p$ a stronger confirmation depth can significantly reduce the

chance that an attacker would reverse a transaction. This is far more compelling than simply claiming that blockchain records are unchangeable.

However, the same calculations also reveal some limitations for regulated settlements: security is conditional and probabilistic. The attacker's hash power and the depth of confirmations heavily influence the probability of security, and the paper provides little empirical evidence for the validity of its assumptions on propagation, mining concentration, strategic behaviour, and network stress. Furthermore, double spending alone is not a guarantee of legal finality, governance, operational resilience, and regulatory accountability.

Therefore, the design should be revised and empirically validated before it is considered for use in regulated remittance infrastructure. This does not mean that Bitcoin is insecure, PoW cannot support payments, or that PBFT/RTGS is universally better. It is more specific than that: Nakamoto (2008) fails to provide sufficient empirical and institutional evidence that its probabilistic consensus mechanism is suitable for regulated remittance settlement.

REFERENCES

Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, 173–186.

Committee on Payment and Settlement Systems, & Technical Committee of the International Organization of Securities Commissions. (2012). *Principles for financial market infrastructures*. Bank for International Settlements.

Decker, C., & Wattenhofer, R. (2013). Information propagation in the Bitcoin network. *2013 IEEE 13th International Conference on Peer-to-Peer Computing*, 1–10.

Gervais, A., Karame, G. O., Wüst, K., Glykantzis, V., Ritzdorf, H., & Čapkun, S. (2016). On the security and performance of proof of work blockchains. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (pp. 3–16). Association for Computing Machinery.

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system*.

Antonopoulos, A.M. and Harding, D.A. (2023) *Mastering Bitcoin: Programming the Open Blockchain*. 3rd edn. Sebastopol, CA: O'Reilly Media.

Eyal, I. and Sirer, E.G. (2014) 'Majority is not enough: Bitcoin mining is vulnerable', in *Financial Cryptography and Data Security*, pp. 436–454.

STATEMENT OF USE OF AI

Claude AI was used in the creation of this assignment to do the following:

- Format jupyter notebook for Part A
- Create Readme.md and Requirements.txt
- Modify Part A code to find system specifications.

- Formatting Final report on Microsoft Word, for report structure and tables.

All outputs were reviewed to ensure accuracy and validity

GITHUB LINK

Link: <https://github.com/Thapel0R/Assignment-7>